

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA0506	Course Title:	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	50 (5 Questions × 10 Mark)
CO1 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4).</i>		
Student Name:	Y.Tejanvesh Reddy	Reg. No.:	192425156
Section / Batch:	D	Date:	

Code of Conduct

I Tejanvesh Reddy.Y-192425156 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10
Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
GRAND TOTAL:			50 Marks

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features

systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

Solution:

Aim :

To compare Harris Corner Detection and SIFT (Scale-Invariant Feature Transform) on the same set of images and analyze their feature quality, robustness, repeatability, and suitability for image matching.

Program :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load image
image = cv2.imread("image.jpg")
if image is None:
    print("Error: Image not found.")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# ----- HARRIS -----
gray_float = np.float32(gray)
harris = cv2.cornerHarris(
    gray_float,
    blockSize=2,
    ksize=3,
    k=0.04
)
harris = cv2.dilate(harris, None)
harris_img = image.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]
# ----- SIFT -----
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
sift_img = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
# Display
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(harris_img, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(sift_img, cv2.COLOR_BGR2RGB))
```

```
plt.title("SIFT Feature Detection")
plt.axis("off")
plt.show()
print("Number of SIFT keypoints:", len(keypoints))
print("Descriptor size:", descriptors.shape if descriptors is not None else None)
```

Output :



2. Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

Solution :

Aim:

To study the effect of histogram equalization and contrast adjustment on feature detection and determine whether enhancement improves feature visibility and detection reliability.

Program :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image
image = cv2.imread("image.jpg")
if image is None:
    print("Error: Image not found.")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Histogram Equalization
equalized = cv2.equalizeHist(gray)
# Contrast Stretching
```

```

contrast = cv2.normalize(
    gray,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)
# SIFT feature detector
sift = cv2.SIFT_create()
kp_original, _ = sift.detectAndCompute(gray, None)
kp_equalized, _ = sift.detectAndCompute(equalized, None)
kp_contrast, _ = sift.detectAndCompute(contrast, None)
# Draw features
original_features = cv2.drawKeypoints(
    gray,
    kp_original,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
equalized_features = cv2.drawKeypoints(
    equalized,
    kp_equalized,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
contrast_features = cv2.drawKeypoints(
    contrast,
    kp_contrast,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
# Display
plt.figure(figsize=(15, 8))
plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(contrast, cmap="gray")
plt.title("Contrast Adjustment")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(original_features, cmap="gray")
plt.title("Original Features")
plt.axis("off")

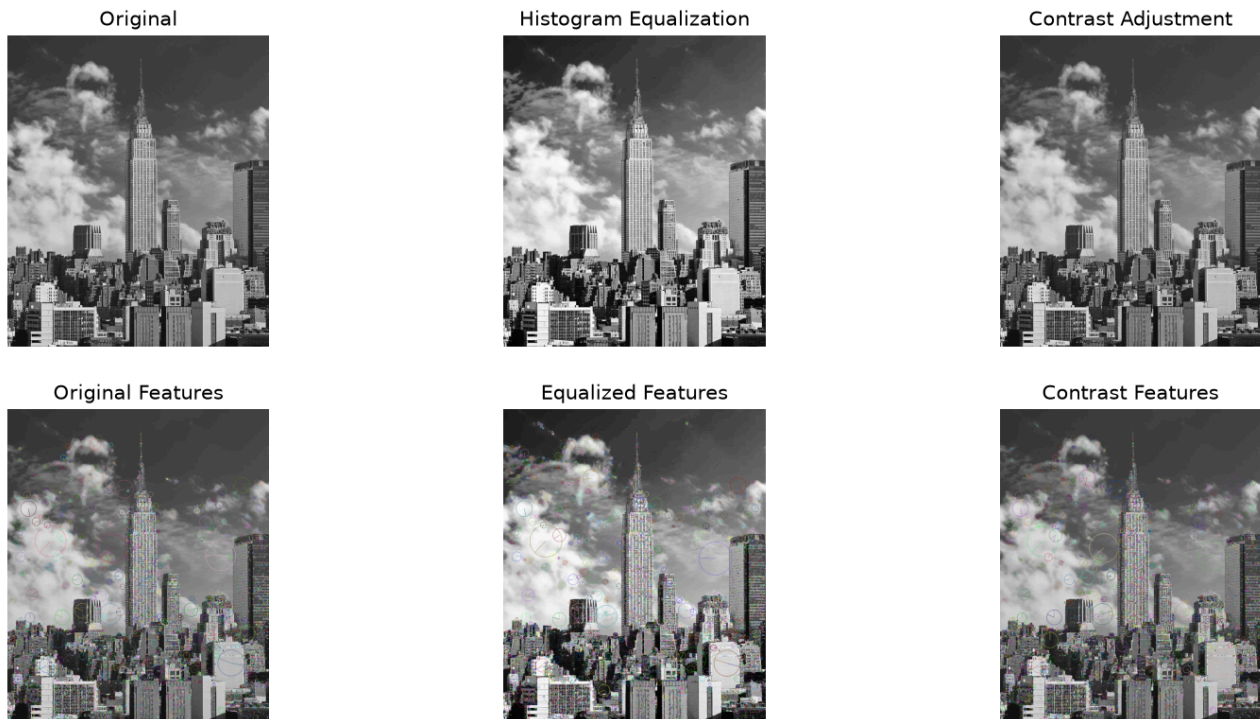
```

```

plt.subplot(2, 3, 5)
plt.imshow(equalized_features, cmap="gray")
plt.title("Equalized Features")
plt.axis("off")
plt.subplot(2, 3, 6)
plt.imshow(contrast_features, cmap="gray")
plt.title("Contrast Features")
plt.axis("off")
plt.show()
print("Original features:", len(kp_original))
print("Equalized features:", len(kp_equalized))
print("Contrast adjusted features:", len(kp_contrast))

```

Output :



3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features

systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

Solution :

Aim :

To study the effect of image scale on feature detection and analyze feature stability, accuracy, and computational performance at different resolutions.

Program:

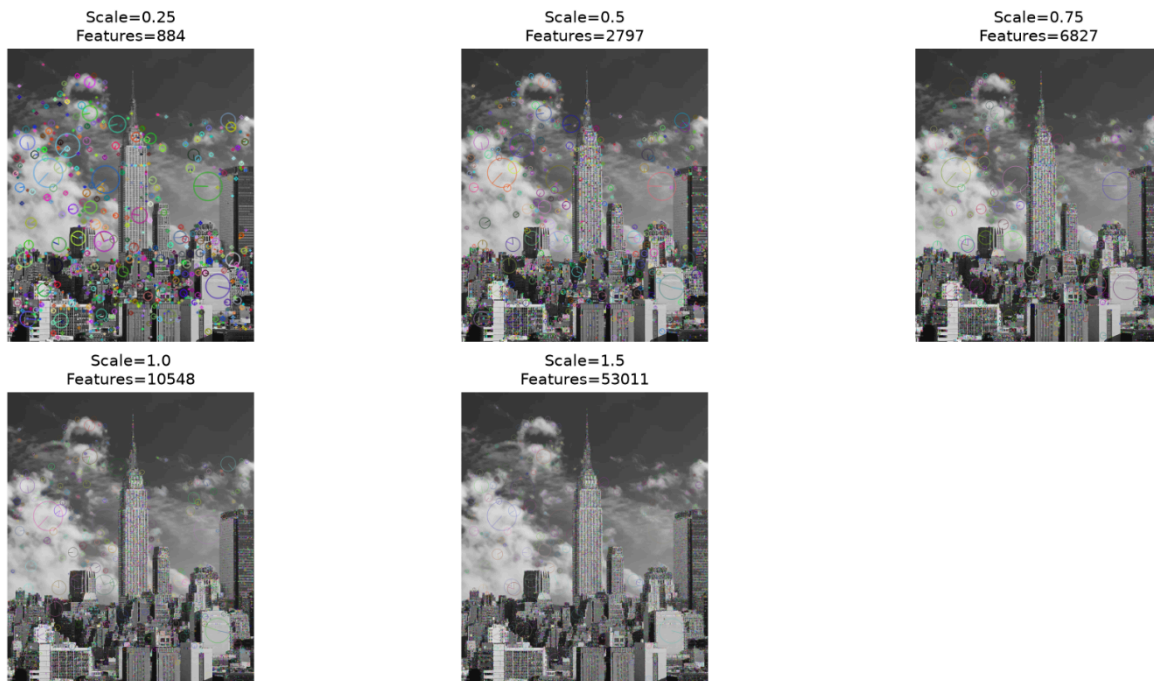
```
import cv2
import time
import matplotlib.pyplot as plt
image = cv2.imread("image.jpg")
if image is None:
    print("Error: Image not found.")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
scales = [0.25, 0.5, 0.75, 1.0, 1.5]
sift = cv2.SIFT_create()
results = []
plt.figure(figsize=(15, 8))
for i, scale in enumerate(scales):
    resized = cv2.resize(
        gray,
        None,
        fx=scale,
        fy=scale,
        interpolation=cv2.INTER_AREA
        if scale < 1
        else cv2.INTER_CUBIC
    )
    start = time.time()
    keypoints, descriptors = sift.detectAndCompute(
        resized,
        None
    )
    end = time.time()
    processing_time = end - start
    results.append(
        (scale, len(keypoints), processing_time)
    )
    output = cv2.drawKeypoints(
        resized,
        keypoints,
        None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )
    plt.subplot(2, 3, i + 1)
    plt.imshow(output, cmap="gray")
    plt.title(
```

```

    f"Scale={scale}\nFeatures={len(keypoints)}"
)
plt.axis("off")
plt.tight_layout()
plt.show()
print("\nScale\tFeatures\tTime")
for scale, features, t in results:
    print(
        f"{scale}\t{features}\t{t:.4f} sec"
    )

```

Output :



4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

Aim:

To compare Mean, Gaussian, and Median filters for noise reduction and determine their effectiveness in preserving important image features.

Program:

```

import cv2
import matplotlib.pyplot as plt
image = cv2.imread("noisy_image.jpg")

```

```

if image is None:
    print("Error: Image not found.")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Mean filter
mean_filter = cv2.blur(gray, (5, 5))
# Gaussian filter
gaussian_filter = cv2.GaussianBlur(
    gray,
    (5, 5),
)
# Median filter
median_filter = cv2.medianBlur(
    gray,
    5
)
# Display
plt.figure(figsize=(14, 8))
plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original / Noisy")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(mean_filter, cmap="gray")
plt.title("Mean Filter")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(gaussian_filter, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(median_filter, cmap="gray")
plt.title("Median Filter")
plt.axis("off")
plt.tight_layout()
plt.show()

```

Output :



5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Aim:

To design and implement a complete object detection pipeline involving:

1. Image preprocessing
 2. Edge detection
 3. Feature extraction
 4. Shape detection
 5. Final object identification
-

Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# -----
# 1. READ IMAGE
# -----
image = cv2.imread("image.jpg")
if image is None:
    print("Error: Image not found.")
    exit()
original = image.copy()
# -----
# 2. RESIZE
# -----
```

```

image = cv2.resize(
    image,
    (800, 600)
)

# -----
# 3. GRAYSCALE
# -----
gray = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)
# -----
# 4. GAUSSIAN FILTER
# -----
blur = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)
# -----
# 5. CONTRAST ENHANCEMENT
# -----
enhanced = cv2.equalizeHist(blur)
# -----
# 6. CANNY EDGE DETECTION
# -----
edges = cv2.Canny(
    enhanced,
    50,
    150
)
# -----
# 7. CONTOUR DETECTION
# -----

contours, hierarchy = cv2.findContours(
    edges,

```

```

    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
output = image.copy()
# -----
# 8. SHAPE DETECTION
# -----
for contour in contours:
    area = cv2.contourArea(contour)
    if area < 500:
        continue
    perimeter = cv2.arcLength(
        contour,
        True
    )
    approximation = cv2.approxPolyDP(
        contour,
        0.04 * perimeter,
        True
    )
    vertices = len(approximation)
    x, y, w, h = cv2.boundingRect(
        approximation
    )
    if vertices == 3:
        shape = "Triangle"
    elif vertices == 4:
        aspect_ratio = w / float(h)
        if 0.90 <= aspect_ratio <= 1.10:
            shape = "Square"
        else:
            shape = "Rectangle"

    elif vertices > 6:
        shape = "Circle"
    else:
        shape = "Unknown"
cv2.drawContours(

```

```

        output,
        [approximation],
        -1,
        (0, 255, 0),
        2
    )
    cv2.putText(
        output,
        shape,
        (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 0, 0),
        2
    )
# -----
# 9. DISPLAY RESULTS
# -----
plt.figure(figsize=(16, 10))
plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(enhanced, cmap="gray")
plt.title("Enhanced Image")
plt.axis("off")
plt.subplot(2, 3, 5)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")

```


Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding; unable to integrate concepts	

Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge
Signature: _____
Date: _____